AI를 어떻게 썼나

프롬프트 방법론 · 문서 체계

발표자 1 · 8분

AI로 쇼핑몰 만들기 — 잘 묻고, 반드시 검증한다

무엇을 만들었나

26	33	110	960
고객 화면	관리자 화면	API	자동 테스트

> 한국형 온라인 쇼핑몰 — 고객몰 · 관리자 · 백엔드

오늘 할 얘기는
“쇼핑몰을 만들었다”가 아닙니다

AI를 어떻게 써서 만들었나

AI 쓰는 방식이 네 번 바뀌었습니다

6월 중순 시작 → 8월 초 · 약 7주

1기

그냥 시켰다

고치는 시간이
짜는 시간보다 길었다

→

2기

공식을 정했다

코드는 좋아졌는데
헛구현은 그대로

→

3기

함정을 미리 심었다

같은 실수를
다시 밟지 않게 됨

→

4기

문서를 나눴다

문서가 쌓여도
정확도가 안 떨어짐

> 처음부터 이렇게 한 게 아닙니다. **각 단계는 앞 단계가 깨져서 생겼습니다.**

1기 — 그냥 시켰습니다

우리가 한 말

상품 페이지 만들어줘

돌아온 것

· 매번 다른 구조
· 프로젝트 규칙 무시
· 로딩 · 빈 화면 · 에러 처리 없음

받아서 고치는 시간이 직접 짜는 시간보다 길었습니다. 여기서 방식을 바꿨습니다

2기 — 프롬프트에 공식을 만들었습니다

좋은 프롬프트 = 5요소

- ① **맥락** — 어떤 프로젝트·스택·디렉터리인지
- ② **할 일 (하나)** — 화면 1개 또는 함수 1개. 여러 개 절대 X
- ③ **따를 패턴** — "옆에 있는 OO처럼" 기존 코드를 지목
- ④ **출력 형태** — 타입 먼저 / diff / 파일:라인
- ⑤ **검증 요구** — "끝나면 타입체크하고 화면 확인까지"

같은 요청, 다른 결과

❌ 나쁜 예

상품 페이지 만들어줘

✅ 좋은 예

`products/[handle]` 페이지에 리뷰 탭 1개 추가해줘.
데이터는 `lib/data/reviews.ts` 경유
(없으면 mock+타입 먼저 제안).
옆 상세 섹션 패턴 그대로 따르고,
로딩/빈/에러 상태 처리.
먼저 접근을 제안하고 내 확인을 받은 뒤 구현해.
끝나면 `npx tsc --noEmit` 로 타입체크까지.

그런데 공식만으로는 안 됐습니다

프롬프트는 좋아졌는데, 크게 시키면 여전히 무너졌다

"주문 워크플로우 전체 만들어줘"

→ 코드는 나왔다
→ 그런데 어디가 틀렸는지 알 수가 없었다
→ 되돌리고 처음부터

덩어리가 크면 **검증할 수가 없습니다.** 그래서 규칙을 세 개 더 붙였습니다

규칙 세 개를 붙였습니다

- **① 접근을 먼저 제안하게** — 바로 코드부터 뽑지 못하게
  30분짜리 삽질을 30초 대화로 막는다
- **② 한 프롬프트 = 한 기능** — 재고 / 할인 / 적립금을 따로
  단계마다 테스트하고 넘어갈 수 있다
- **③ 계약(타입)을 먼저** — 타입 → mock → 화면 → 실 API
  타입만 맞춰두면 서로 안 기다려도 된다

> ①은 헛구현을, ②는 "검증 불가"를, ③은 서로 기다리는 시간을 없앴습니다

3기 — 겪은 실수를 프롬프트에 심었습니다

고친 뒤 그냥 넘어가지 않고, 다음 프롬프트에 경고로 박았다

```
mock 에 Math.random() · new Date() 쓰지 마
   → 하이드레이션 에러가 난다

items.total 같은 계산 필드를 단독 선택하지 마
   → null 이나 절반 값으로 온다

dev 서버 켠 채로 build 하지 마
   → 실행 중인 화면이 500 이 된다
```

> AI는 같은 함정을 **매번 다시 밟습니다.** 한 번 겪을 때마다 문서에 적고 프롬프트에 붙였더니 안 밟더군요

4기 — 문서를 AI에게 줬더니 결과가 나빠졌습니다

문서가 70개까지 쌓인 시점

사람용 문서

"왜 그렇게 했나"
비유 · 배경 · 맥락
읽기 좋게

AI용 문서

"정확히 무엇인가"
타입 · 경로 · 제약
토큰 효율

> 비유와 배경이 토큰만 잡아먹고, 정작 필요한 타입 정의는 묻힌다

★ 문서를 독자별 4층으로 나눴습니다

폴더	독자	형식
01_팀원용	사람 (합류자)	설명 · 비유 · "왜 그렇게 했나"
02_AI용	AI	사실 나열 · 정확한 타입과 경로
03_운영용	사장님 · 운영자	개발 지식 0 가정
04_점검기록	검증하는 사람	결과 + 재현 가능한 실행 도구

> 02_AI용 문서는 "이 문서를 읽는 AI는 …"으로 시작한다. 사람이 읽으라고 쓴 게 아니다

★ 네 번 바꾸고 나서

바꾼 것	달라진 것
1기 → 2기 프롬프트 5요소	프로젝트 규칙을 지킨 코드가 나오기 시작
2기 → 3기 접근 먼저 · 한 기능씩 · 계약 먼저	되돌리는 일이 줄고, 병렬 작업이 가능해짐
3기 → 4기 겪은 함정을 프롬프트에 주입	커스텀 모듈 **27개가 같은 구조**로 유지됨
4기 문서를 독자별 4층으로	문서 **70개**를 쌓아도 답변 정확도가 안 떨어짐

> 앞 두 줄은 **체감**이고, 뒤 두 줄은 **결과물로 확인**됩니다 — 모듈 27개 · 문서 4층

정리

- **처음부터 잘 쓴 게 아닙니다** — 네 번 깨지고 네 번 바꿨습니다
- **깨질 때마다 규칙으로 남겼습니다** — 사람이 기억하는 게 아니라 프롬프트·문서에
- **다음 두 분이 이 방식으로 만든 것과 검증한 것을 보여드립니다**